Multi-vehicle Navigation

1. Course Content

- Run example programs, give different navigation points to two vehicles in rviz. The two vehicles will plan their respective navigation paths based on their positions in the map, navigate to the specified locations, and avoid obstacles in real-time during navigation.

1.1 Preparation Work

Refer to the method in 【1. Multi-vehicle Control】 to set the vehicle's `namespace` and `DOMAIN_ID`

1.2 Navigation Map

Before starting multi-vehicle navigation, you need to place the map files in the `~/map/` path of the vehicle that starts the map service. The map files include .yaml format parameter files and .pgm format image files.

1.3 Setting Vehicle Navigation Name

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

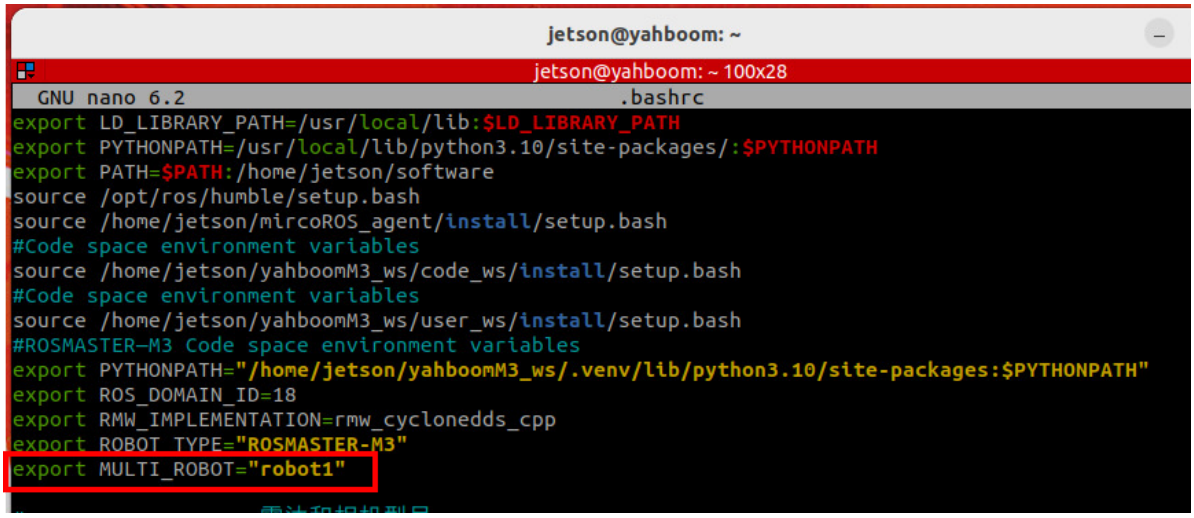
```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

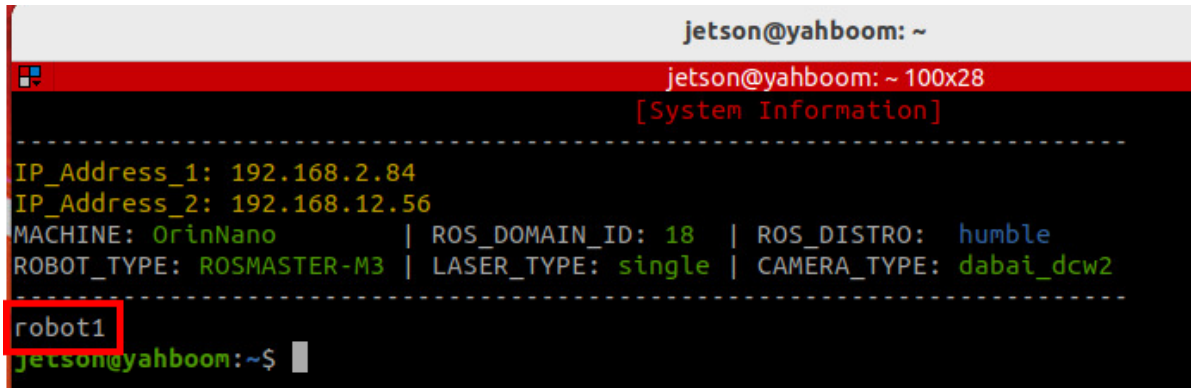
- Edit the `MULTI_ROBOT` environment variable in .bashrc

```
nano .bashrc
```



- Modify the `MULTI_ROBOT` environment variable to be the same as the vehicle's `namespace` set earlier. Here we use `robot1` as an example, and other vehicles follow this pattern.
- After modification, press ctrl+x to save and exit
- Refresh the terminal variables and verify if the setting is successful

```
source .bashrc
echo $MULTI_ROBOT
```



2. Starting Example Programs

1. You need one vehicle to run the map service loading service and RVIZ visualization interface. This vehicle must place the map files in the `~/map/` path.
2. Run commands on the vehicle terminal that starts visualization and map service:

- **(ROBOT1)** (ORIN mainboard)

```
ros2 launch m3_bringup multi_nav2.launch.py start_mapserve:=True gui:=True
```

(RDK & PI5 mainboard) run in steps, visualization starts on the paired virtual machine `Rosmaster-M3-Ubuntu22.04`,

```
#Virtual machine side
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/multi_nav2.rviz
#Host side
ros2 launch m3_bringup multi_nav2.launch.py start_mapserve:=True gui:=True
use_rviz:=False
```

- Other vehicles **(ROBOT2/3)** directly run multi-vehicle navigation nodes without startup parameters

(ORIN mainboard)

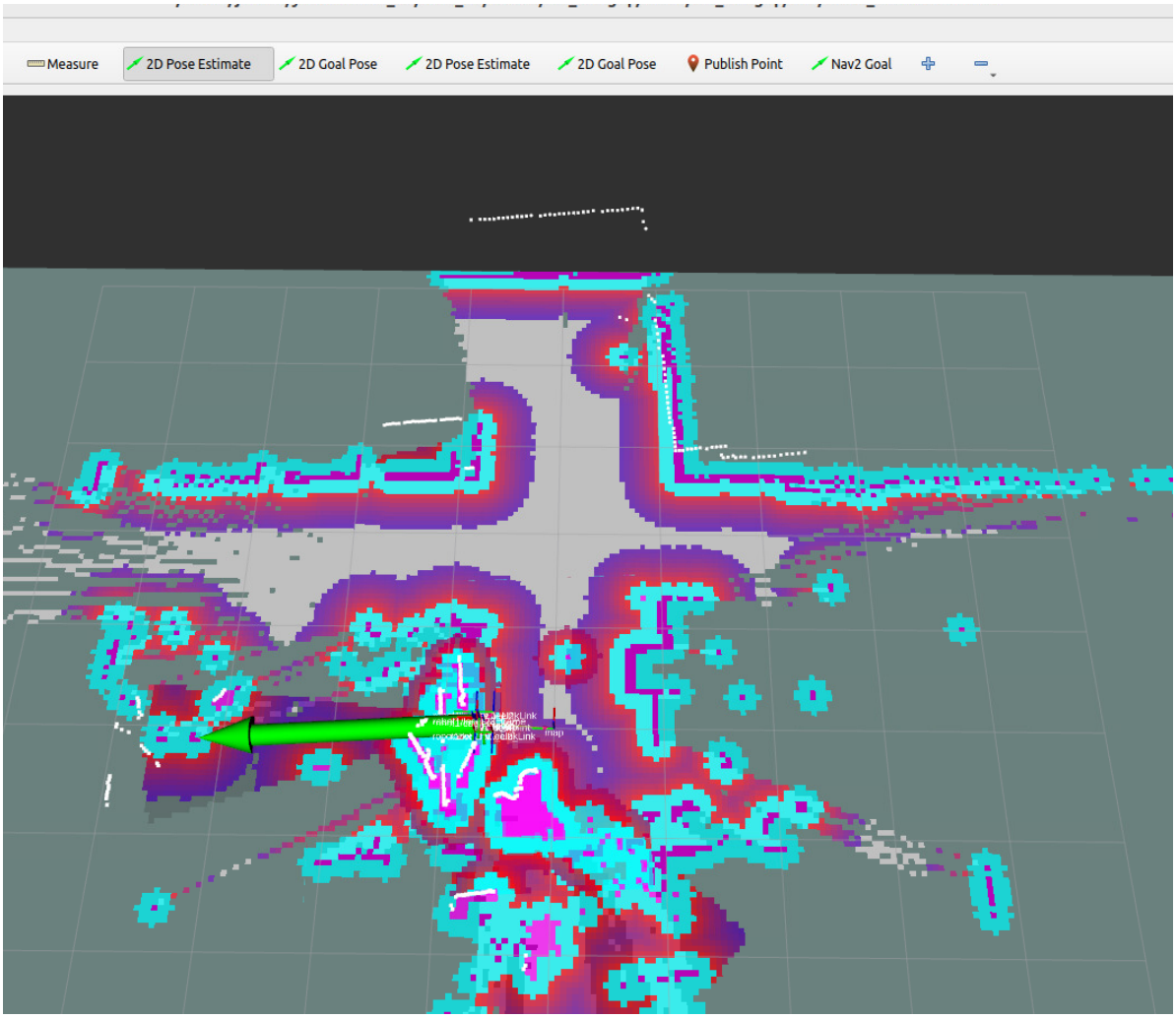
```
ros2 launch m3_bringup multi_nav2.launch.py
```

(RDK & PI5 mainboard)

```
ros2 launch m3_bringup multi_nav2.launch.py use_rviz:=False
```

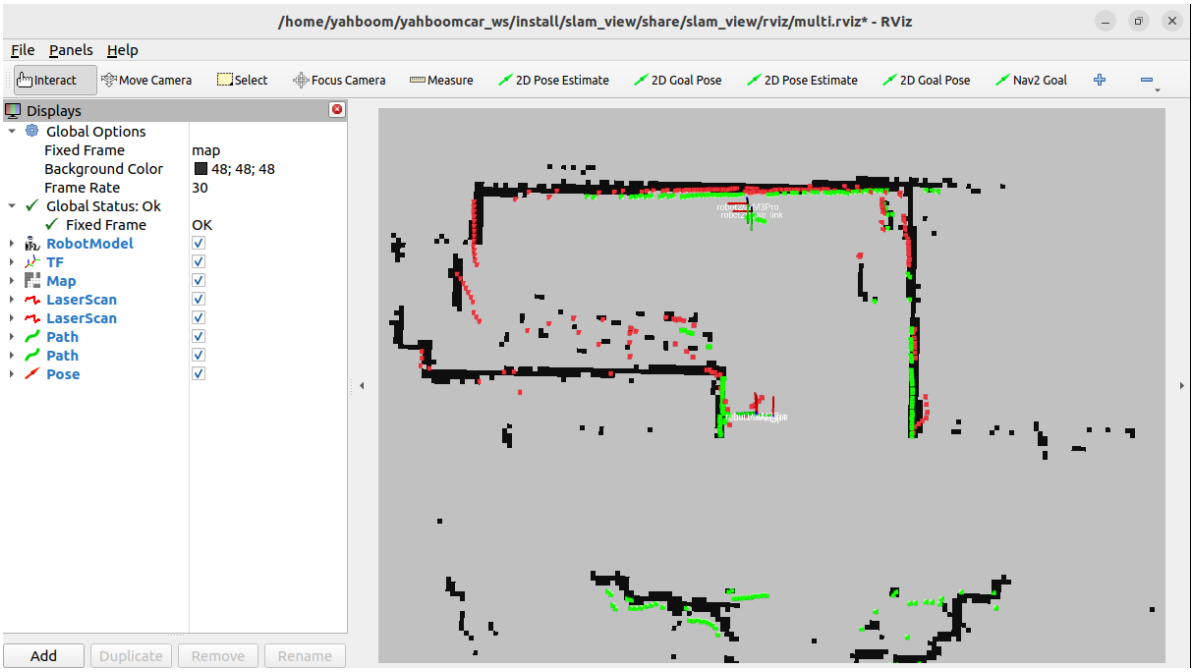
> [!NOTE]
>
> **Optional startup parameter descriptions:**
>
> - start_mapserve: Start map server to load map, only one vehicle needs to start it, default `False`
> - use_rviz: Start rviz2 visualization interface, only one vehicle needs to start it, default `False`
> - gui: Start rtabmap visualization point cloud, default `False`
> - map: Name of the map file to be loaded, map must be saved in the system `~/map/` path, default is `map.yaml`

2.1 Initialize Vehicle Pose



- Use the 【2D Pose Estimate】 tool to give the vehicle an initial pose. There are two 【2D Pose Estimate】 tools in rviz. The first is the tool for giving robot1's initial pose.
- The second is the tool for giving robot2's initial pose. Use these two tools respectively to give initial poses to the two vehicles.

As shown in the figure below, make the area scanned by the two radars overlap with the black areas on the map. Among them, the green point cloud is the radar scan of robot1 vehicle, and the red point cloud is the radar scan of robot2 vehicle.



Next, in rviz2, use the 【2D Goal Pose】 tool to give the vehicle an initial pose. There are two 【2D Goal Pose】 tools in this rviz. The first is the tool for giving robot1's target point, and the second is the tool for giving robot2's target point. Use these two tools respectively to give target points to the two vehicles. As shown in the figure below, after the vehicle is given the target point, it will plan a path and navigate to the target point.

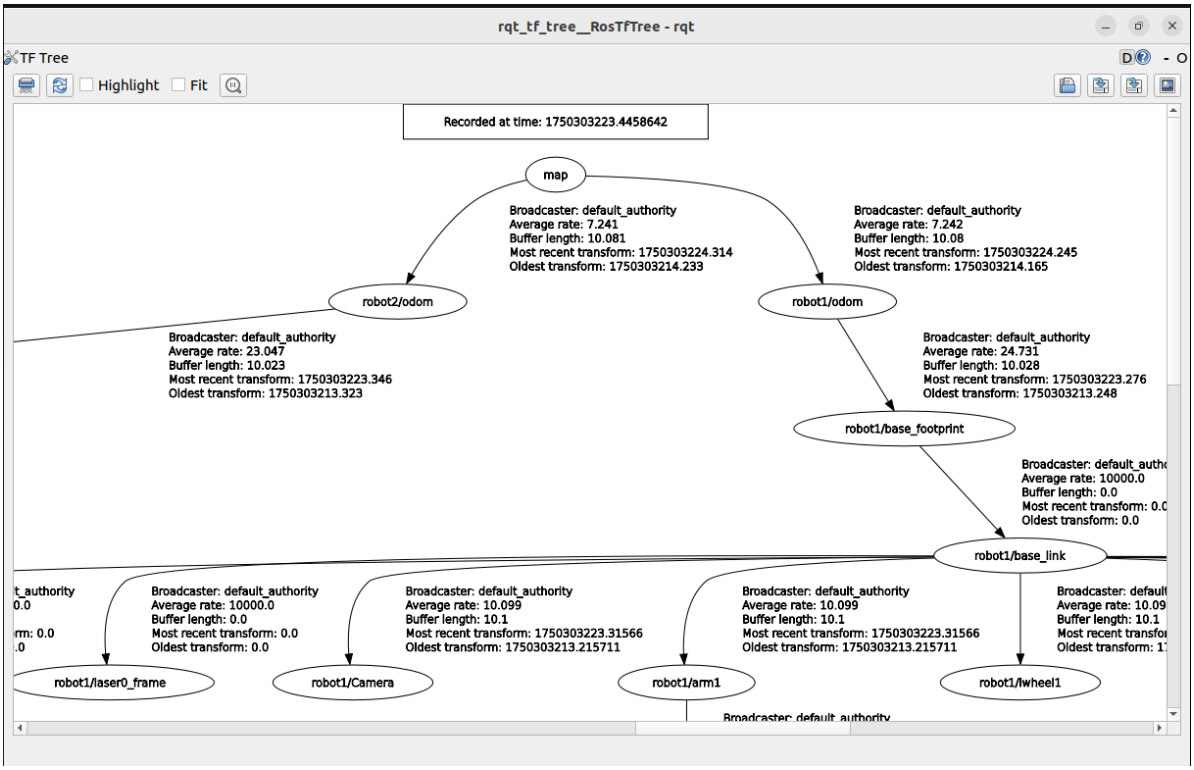
3. Source Code Analysis

3.1 View TF Coordinate Tree

- Enter the following command in the vehicle terminal to view the TF tree,

```
ros2 run rqt_tf_tree rqt_tf_tree
```

As shown in the figure below, this is the TF tree for multi-vehicle navigation



3.2 Launch Files

- Configuration file directory:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/multi_robot
```

Contains the following files:

```
├── multi_amcl_param.yaml                Navigation AMCL localization parameter
file
├── multi_common_param.yaml             Sensor configuration parameter file
├── multi_nav_param.yaml                Navigation Nav2 configuration parameter
file
└── multi_robot_localization.yaml        Odometer fusion configuration parameter
file
```

- Launch source code path

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/multi_robot/multi_nav2.launch.py
```

- Core part of the top-level launch file
- This file will sequentially start the basic nodes of carbase robot sensors and odometer, map_serve map service node, nav2_stack navigation-related nodes, and rviz_node visualization display node, and distinguish which specific nodes to start through startup parameters.

```
def generate_launch_description():
    MULTI_ROBOT = os.environ.get('MULTI_ROBOT', '')
```

```
    if MULTI_ROBOT == '':
        raise Exception(Fore.RED+'MULTI_ROBOT is not set!!!!'+Fore.RESET)
    m3_bringup_dir=os.path.join(get_package_share_directory('m3_bringup'))
    nav_rviz_config=os.path.join(m3_bringup_dir,'rviz','multi_nav2.rviz')

    #Declare arguments
    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "map",
            default_value="map.yaml",
            description="map name",
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "gui",
            default_value='False',
            description="if rviz is used, set to True. If False, rviz will not
be launched.",
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "start_mapserve",
            default_value='False',
            description="if map server is used, set to True. If False, map
server will not be launched.",
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "use_sim_time",
            default_value="False",
            description="Use simulation (Gazebo) clock if true",
        )
    )

    #Initialize Arguments
    map = LaunchConfiguration("map")
    map_path=PathJoinSubstitution([map_folder_dir, map])
    gui = LaunchConfiguration("gui")
    start_mapserve=LaunchConfiguration("start_mapserve")

    use_sim_time=LaunchConfiguration("use_sim_time",default="false")

    #navigation2 startup file
    nav2_launch_file = os.path.join(get_package_share_directory("m3_bringup"),
                                    "launch",
                                    'multi_robot',
                                    'multi_bringup.launch.py')

    #Basic nodes such as robot models and radars
    carbase=ExecuteProcess(
                cmd=['ros2', 'launch',
'm3_bringup','multi_car_base.launch.py'],output='screen')
```

```
    # Navigation2 stack
    nav2_stack=TimerAction(
                period=5.0,
                actions=[
                    IncludeLaunchDescription(
                        PythonLaunchDescriptionSource(nav2_launch_file))
                ]
            )

    amcl_localization=ExecuteProcess(
                cmd=['ros2', 'launch',
'm3_bringup','multi_amcl.launch.py'],output='screen')

    #map service
    map_serve = GroupAction(
        condition=IfCondition(PythonExpression(["'", start_mapserve, "' ==
'True'" ])),
        actions=[
            Node(
                package='nav2_map_server',
                executable='map_server',
                name='map_server',
                output='screen',
                respawn_delay=2.0,
                parameters=[{'yaml_filename': map_path}],
                arguments=['--ros-args', '--log-level', "info"],
                remappings=[('/tf', 'tf'),('/tf_static', 'tf_static')]),
            Node(
                package='nav2_lifecycle_manager',
                executable='lifecycle_manager',
                name='lifecycle_manager_map',
                output='screen',
                arguments=['--ros-args', '--log-level', "info"],
                parameters=[{'use_sim_time': use_sim_time},
                            {'autostart': True},
                            {'node_names': ['map_server']}])
                ]
    )

    #rviz node
    rviz_node = Node(
        package = 'rviz2',
        executable = 'rviz2',
        name='rviz2',
        arguments=['-d', nav_rviz_config],
        parameters=[{'use_sim_time': use_sim_time}],
        output='screen',
        condition=IfCondition(PythonExpression(["'", gui, "' == 'True'"]))
    )
```

- Other included underlying launch file directories:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/launch/multi_robot/
```

```
├── multi_amcl.launch.py              AMCL localization node launch file
├── multi_bringup.launch.py          Nav2 navigation node launch file
├── multi_car_base.launch.py          Robot sensor and odometer launch file
├── multi_display.launch.py          Robot model launch file
├── multi_laser.launch.py            Laser radar sensor launch file
├── multi_merge_odom.launch.py        Odometer fusion launch file
└── multi_nav2.launch.py              Multi-vehicle navigation top-level launch
file
```